
Data Visualization

...

POLI3148 Data Science in Politics and Public Administration

Dr. Chen Haohan

Agenda

- Logistics
 - In-class exercise 4
 - Assignment 1 (15%) will be assigned this week
 - Final project
 - Reading materials on text analysis
- Data Visualization
 - 道: Philosophy of Data Visualization
 - 策: Strategies of Data Visualization
 - 术: Data Visualization Tools

Philosophy of Data Visualization

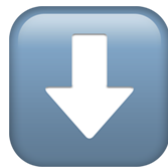
Here are two questions to you:

- What is data visualization?
 - Why do we (you) care about data visualization?
-

Philosophy of Data Visualization: What is it?

Had Gemini make
this.
Not very helpful :(

Numbers



Graphics



Philosophy of Data Visualization: Why do you need it?

“A picture is worth a thousand ~~words~~ number”

There are some scientific basis of it

- Evolution
- How our brain/ memory works

Humans are just biologically and mentally *willing to* and *good at* taking visual information!



Philosophy of Data Visualization: Why do you need it?

For you as a data analyst

- **Summarize** data and results from analysis
- **Discover** from data and data analysis

For you as a writer and presenter

- **Showcase** rigor and complexity
- **Highlight** insights
- **Simplify** interpretation

For you as a consumer

- **Get** the point quickly
 - **Catch** misinterpretation (intentional or not)
 - **Inquire** the right information
- 

Philosophy of Data Visualization

Good data visualization:

- Showcase complexity, not over-complicate
- Highlight, not lie
- Simplify, not over-simplify

Importantly,

Good data visualization $\neq$ **Beautiful** data visualization

Map skills into our data science workflow

- **Data Input:** Transform data for the data visualization tool
- **Data processing**
 - 1 quantitative variable
 - 1 categorical variable
 - 2+ quantitative variables
 - quant. + categorical variables
 - 2+ categorical variables
 - Fancy stuff
 - Good practice
- **Data Output:** Save and export the plot



Steps for successful data visualization

- **Decide** what kind of visualization you need
 - **Identify** the right visualization tool
 - **Transform** the data to shapes that the tool can work with
 - **Apply** the visualization tool
 - **Export** the visualization (for presentation, report, etc.)
-

Collection of Tools We Focus on seaborn

- Seaborn documentation: <https://seaborn.pydata.org/>
- Created by Dr. Michael Waskom, a data scientist and neuroscientist
<https://mwaskom.github.io/software.html>
- The good
 - User-friendly
 - Have functionality for daily use
 - Many data scientists use it → large community → easy to search for information
- The limitations
 - Limited customization (user-friendliness usually comes with this trade-off)
 - Want customization? Tweak with the library it is built on

Customization: An analogy



Seaborn



Matplotlib

Matplotlib matplotlib

- The “gears” and “engines” for data visualization with Python
- Documentation: <https://matplotlib.org/stable/gallery/index>
- Extremely powerful and flexible, but
 - No nice “default” setup: shapes, colors, themes, etc.
 - Need long and complex code to make figure nice
- Seaborn is built on Matplotlib
 - Functions are made with Matplotlib functionalities
 - Set nice default colors, themes, shapes, etc.
- You can customize Seaborn-based visualization by adding additional Matplotlib-based code

So, data visualization with Seaborn always look like...

- Need to import both the **seaborn** and the **matplotlib** libraries

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

- Use seaborn functionalities to make figures
 - Use matplotlib functionalities to improve the figure
-

Other Python Data Visualization Libraries

- Plotnine: if you used R and ggplot before, this will be familiar
<https://plotnine.org/>
- Interactive plots with Python
 - Plotly: Interactive data visualization tool: <https://plotly.com/python/>
 - Bokeh: Interactive data visualization tool: <https://bokeh.org/>
- Non-Python solution: Google Looker Studio:
<https://lookerstudio.google.com/gallery>

Don't get lost in tools. Tools come and go.

Good data visualization needs tools but is more than just applying tools.

Applying the Right Data Visualization Tool

Two questions to ask to apply technically correct data visualization

- **What are the types of variables to visualize?**
 - Quantitative variables: continuous, count
 - Categorical variables: binary, ordinal, nominal
 - ** complex text, audio, visual data are operationalized as the above types for visualization*
- **How many variables to visualize in one figure?**
 - 1
 - 2
 - 3
 - More

Visualize the World

Get insight from socioeconomic indicators

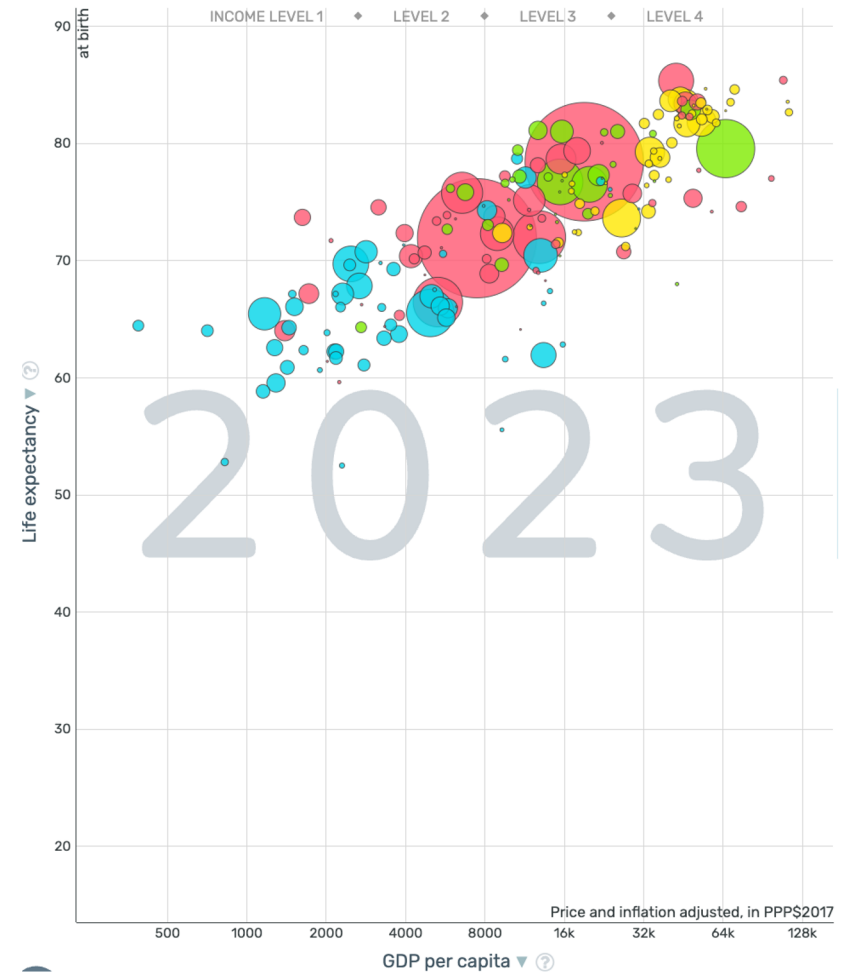
Hans Røling's Gapminder

<https://www.gapminder.org/tools/>

(Check it out!)

Focus: Wealth and Health

- Development: GDP per capita
- Health: Life expectancy



Aside: Other websites similar to Gapminder

- Our World in Data: <https://ourworldindata.org/>
- V-Dem data visualization tools: <https://v-dem.net/graphing/graphing-tools/>

... and more. Many data provider make available in-house visualization tools.

Some are helpful

Variables from V-Dem we Need

10.3	Economics	395
10.3.1	Exports (E) (e_cow_exports)	395
10.3.2	Imports (E) (e_cow_imports)	396
10.3.3	GDP (E) (e_gdp)	396
10.3.4	GDP per capita (E) (e_gdppc)	396
10.3.5	Inflation (E) (e_miinflat)	396
10.3.6	Population (E) (e_pop)	396
10.6	Demography	397
10.6.1	Fertility rate (E) (e_miferrat)	397
10.6.2	Population total (E) (e_mipopula)	398
10.6.3	Urbanization (E) (e_miurbani)	398
10.6.4	Urban population (E) (e_miurbpop)	398
10.6.5	Child mortality rate (E) (e_pechmor)	398
10.6.6	Life expectancy, female (E) (e_pegeliex)	398
10.6.7	Life expectancy (E) (e_pelifeex)	399
10.6.8	Maternal mortality rate (E) (e_pematmor)	399
10.6.9	Population (E) (e_wb_pop)	399

Start with the Year 2019 (the last pre-COVID year!)

Subset to 2019

```
df_s = df.loc[df.year == 2019,:]
```

Get summary statistics (***always*** the recommended first step!)

```
df_s.e_gdppc.describe()
```

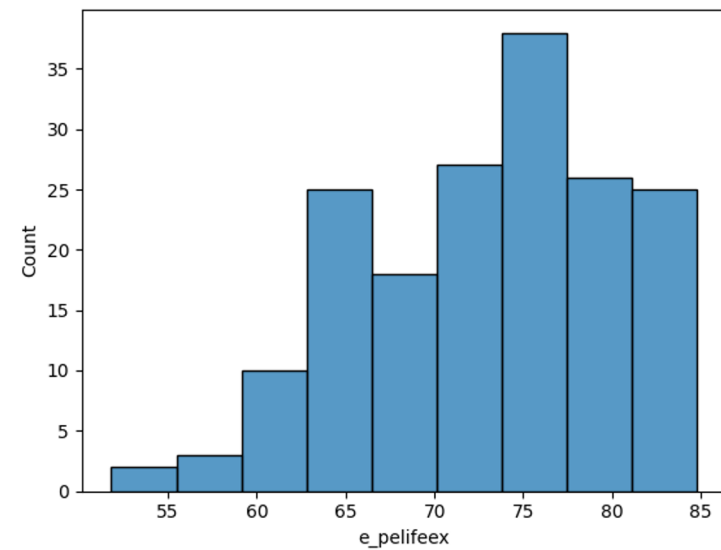
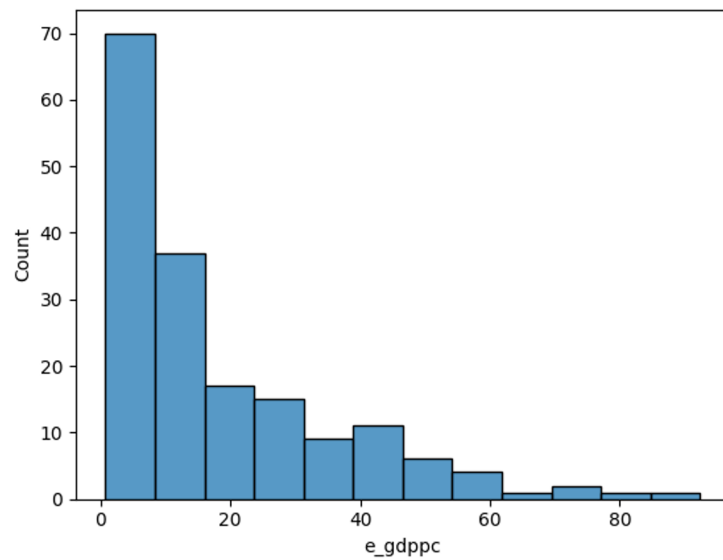
```
df_s.e_pelifeex.describe()
```

1 Quantitative Variable

1 Quantitative Variables: Histogram (Kaggle)

```
sns.histplot(data = df_s, x = "e_gdppc")
```

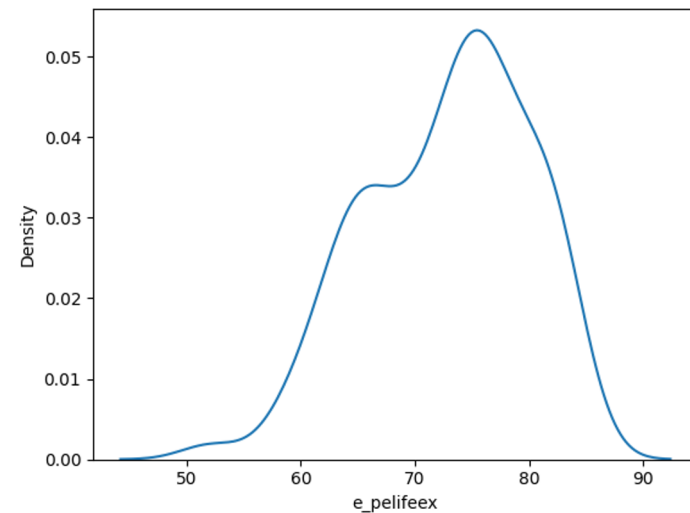
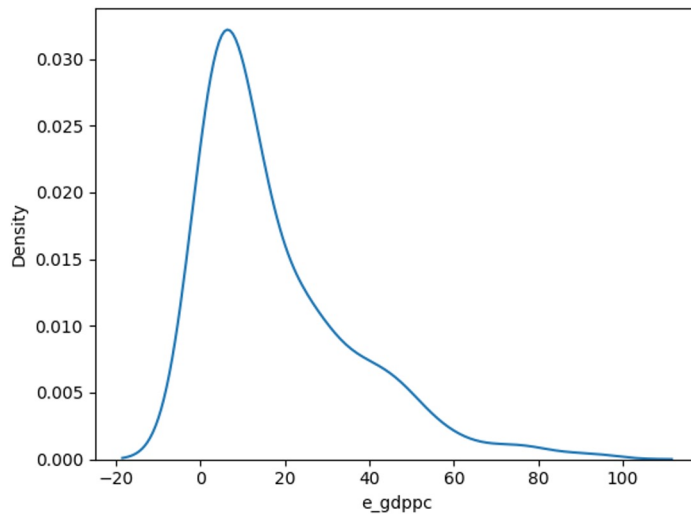
```
sns.histplot(data = df_s, x = "e_pelifeex")
```



1 Quantitative Variable: Density Plot (Kaggle)

```
sns.kdeplot(data = df_s, x = "e_gdppc")
```

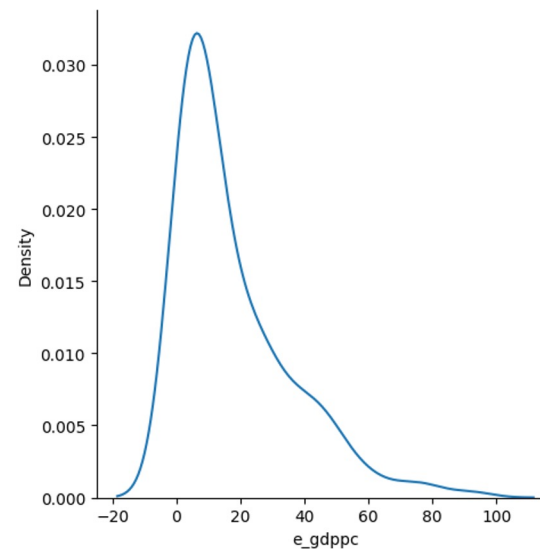
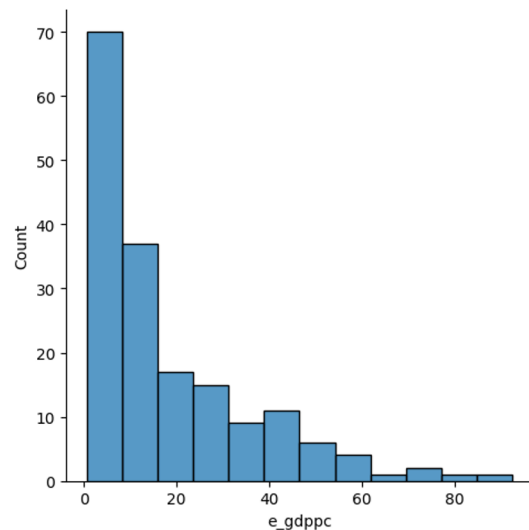
```
sns.kdeplot(data = df_s, x = "e_pelifeex")
```



1 Quantitative Variable: An Unified Alternative -- **displot** (1)

```
sns.displot(data = df_s, x = "e_gdppc")
```

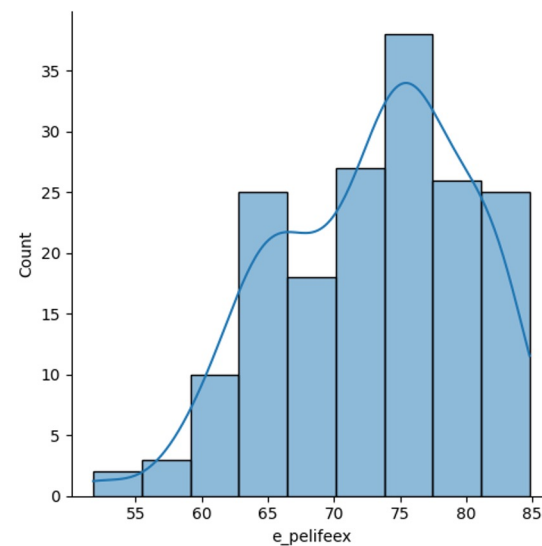
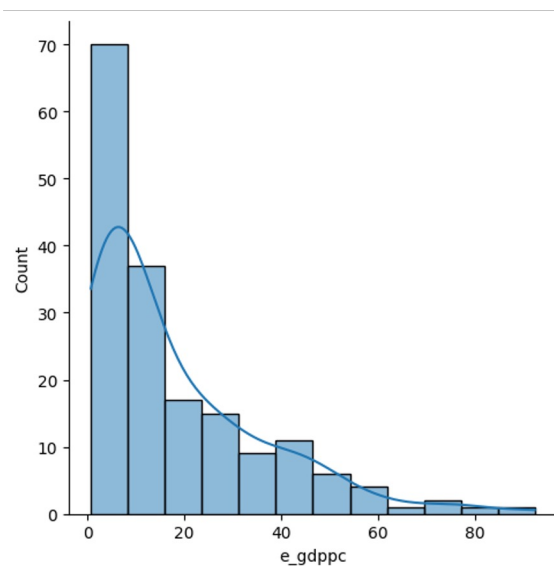
```
sns.displot(data = df_s, x = "e_gdppc", kind = "kde")
```



1 Quantitative Variable: An Unified Alternative -- **displot** (2)

```
sns.displot(data = df_s, x = "e_gdppc", kde = True)
```

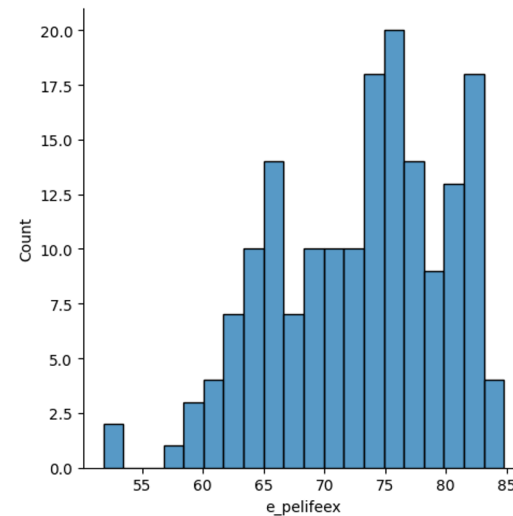
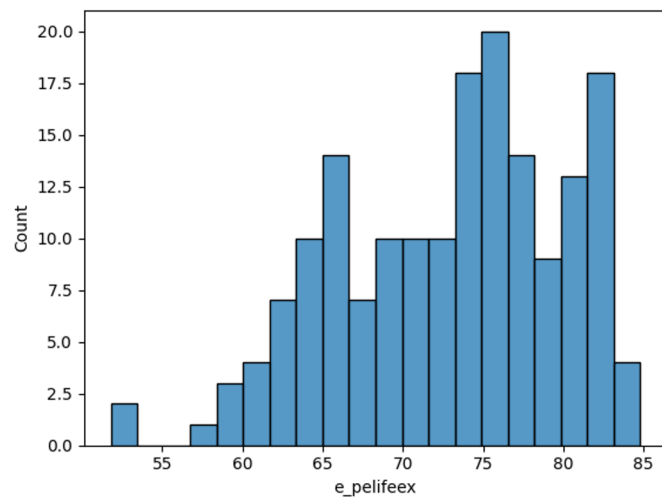
```
sns.displot(data = df_s, x = "e_pelifeex", kde = True)
```



Parameters: Number of bins for histograms

```
sns.histplot(data = df_s, x = "e_pelifeex", bins = 20)
```

```
sns.displot(data = df_s, x = "e_pelifeex", bins = 20)
```



Good practice

For quantitative variables, before any analysis

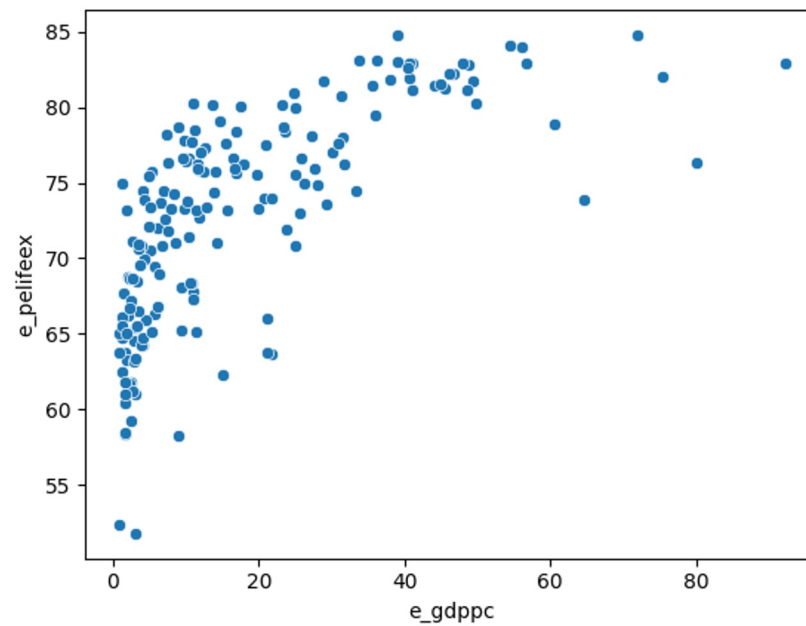
- Always get the summary statistics (mean, variance, quantiles etc.)
- Always plot the histograms

They need not be in your main presentation/ report/ paper, but they should be done and should be made available for readers.

2 Quantitative Variable

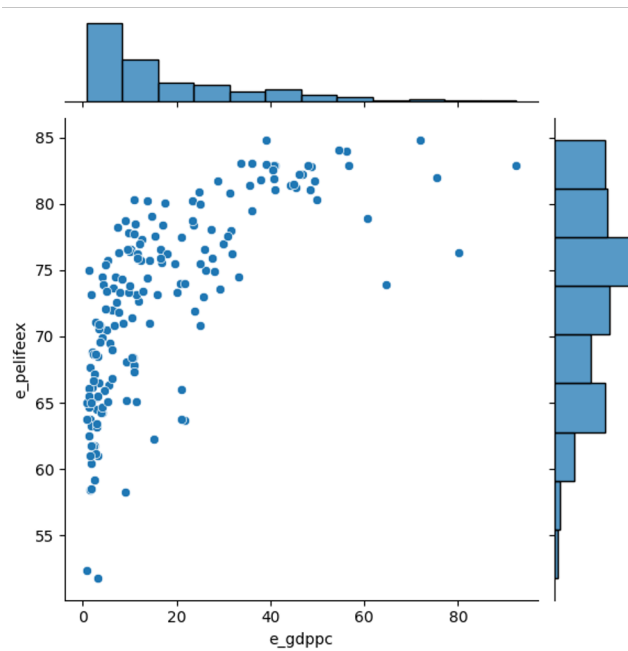
Scatter Plot

```
sns.scatterplot(data = df_s, x = "e_gdppc", y = "e_pelifeex")
```



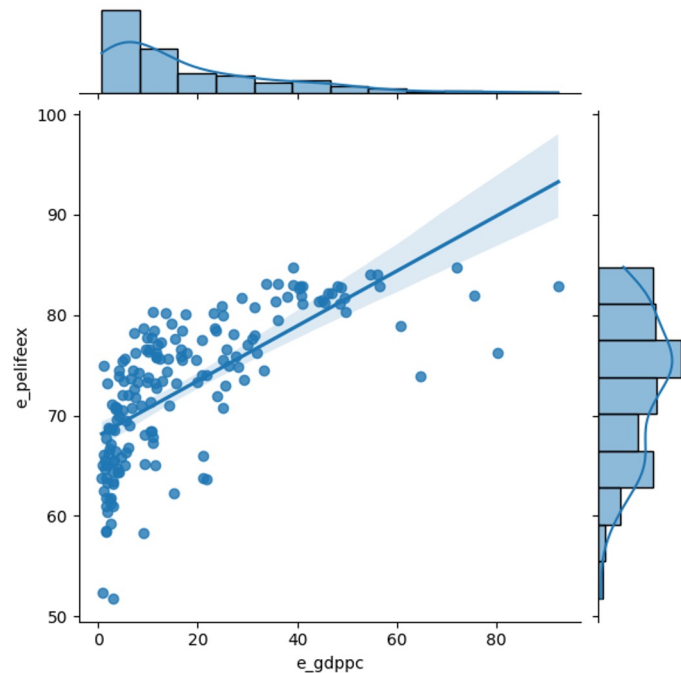
Scatter Plot + Histograms

```
sns.jointplot(data = df_s, x = "e_gdppc", y = "e_pelifeex")
```



Scatter Plot + Histograms + “Trend line” (recommended)

```
sns.jointplot(data = df_s, x = "e_gdppc", y = "e_pelifeex", kind = "reg")
```



Does the association look convincing with the fitted line from regression?

Yes.

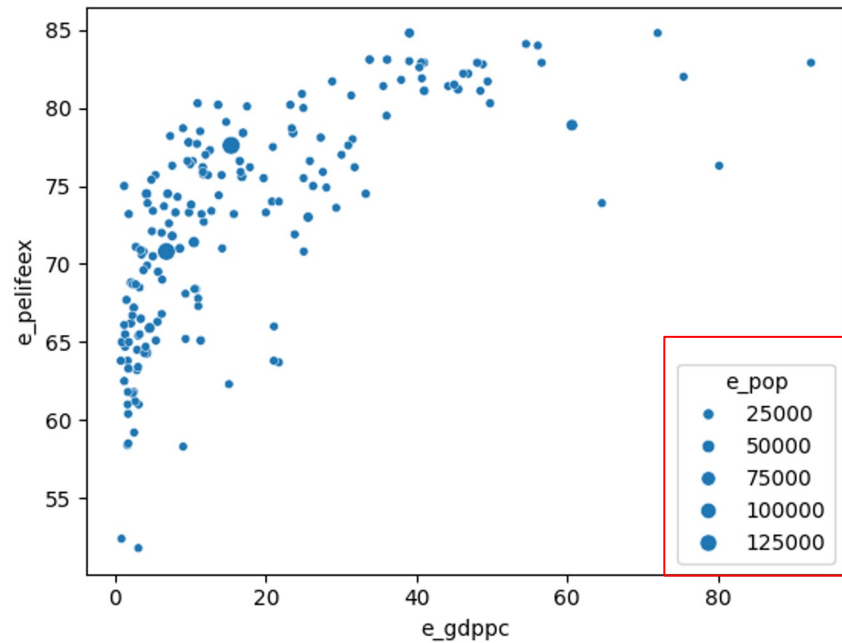
But this only serve to **assist** your pattern discovery. It should not replace rigorous machine/ statistical modeling.

If you find the two quantitative variables to have non-linear correlation, it is not a good idea to add a linear fitted line in your data visualization

3 Quantitative Variables

“Bubble Plots” (1)

```
sns.scatterplot(data = df_s, x = "e_gdppc", y = "e_pelifeex", size = 'e_pop')
```



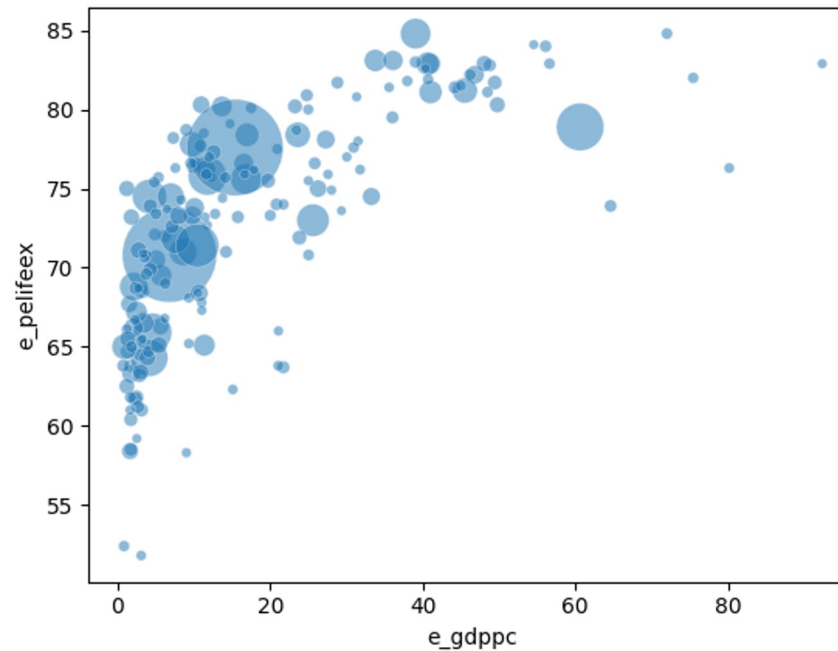
This is called a
“legend”

“Bubble Plots” (2)

Min and max
point sizes

Hide legend

```
sns.scatterplot(data = df_s, x = "e_gdppc", y = "e_pelifeex", size = 'e_pop', sizes = (20, 2000), legend = False, alpha = 0.5)
```



Points'
transparency
(avoid overlap)
0 to 1

1 Categorical Variable

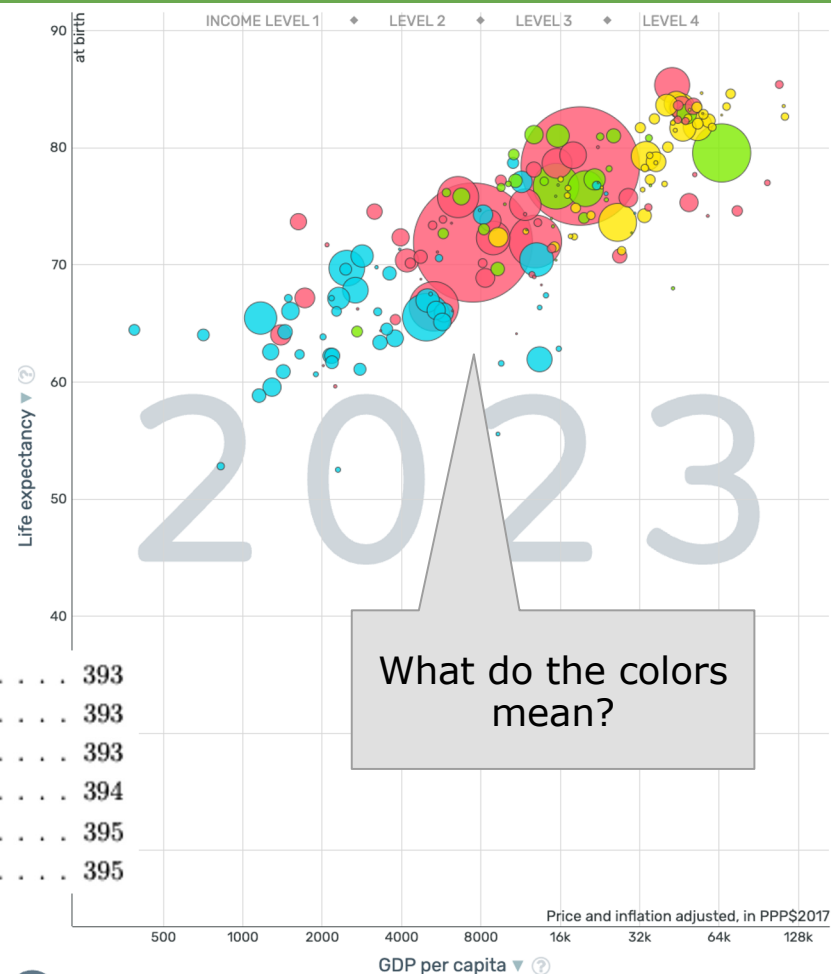
(Prep: Create categorical variables)

Example: Region (1)

Take a look at the data

Regions

10.2	Geography	393
10.2.1	Land area (E) (e_area)	393
10.2.2	Region (geographic) (E) (e_regiongeo)	393
10.2.3	Region (politico-geographic) (E) (e_regionpol)	394
10.2.4	Region (politico-geographic 6-category) (E) (e_regionpol_6C)	395
10.2.5	Region (politico-geographic 7-category) (E) (e_regionpol_7C)	395



Example: Region (2)

10.2.2 Region (geographic) (E) (e_regiongeo)

Question: In which geographic region is this country located?

TOC

393

BACKGROUND FACTORS (E)
10.2 GEOGRAPHY



Clarification: Regions are described based on geographic location.

Responses:

- 1: Western Europe
- 2: Northern Europe
- 3: Southern Europe
- 4: Eastern Europe
- 5: Northern Africa
- 6: Western Africa
- 7: Middle Africa
- 8: Eastern Africa
- 9: Southern Africa
- 10: Western Asia
- 11: Central Asia
- 12: Eastern Asia
- 13: South-Eastern Asia
- 14: Southern Asia
- 15: Oceania (including Australia and the Pacific)
- 16: North America
- 17: Central America
- 18: South America
- 19: Caribbean (including Belize, Cuba, Haiti, Dominican Republic and Guyana)

Source(s): United Nations Statistics Division (2013).

Notes: For the countries coded only in the historical project or for which the UN does not have the code, the region is coded by V-Dem Data Manager in accordance with the position of the neighboring countries.

Data release: 5-14.

Citation: United Nations Statistics Division (2013).

Years: 1789-2023

Make a categorical “region” indicator

```
df_s.loc[:, 'region'] = None
```

```
df_s.loc[df_s.e_regiongeo.isin((1, 2, 3, 4)), 'region'] = "Europe"
```

```
df_s.loc[df_s.e_regiongeo.isin((5, 6, 7, 8, 9)), 'region'] = "Africa"
```

```
df_s.loc[df_s.e_regiongeo.isin((10, 11, 12, 13, 14)), 'region'] = "Asia"
```

```
df_s.loc[df_s.e_regiongeo == 15, 'region'] = "Oceania"
```

```
df_s.loc[df_s.e_regiongeo.isin((16, 17, 18, 19)), 'region'] = "Americas"
```

Check the newly-made categorical indicator of region

```
df_s.e_regiongeo.value_counts()
```

```
df_s.region.value_counts()
```

```
pd.crosstab(df_s.e_regiongeo, df_s.region)
```

Good practice: You should **always** tabulate categorical variables before conducting analysis.

Create income-level indicator

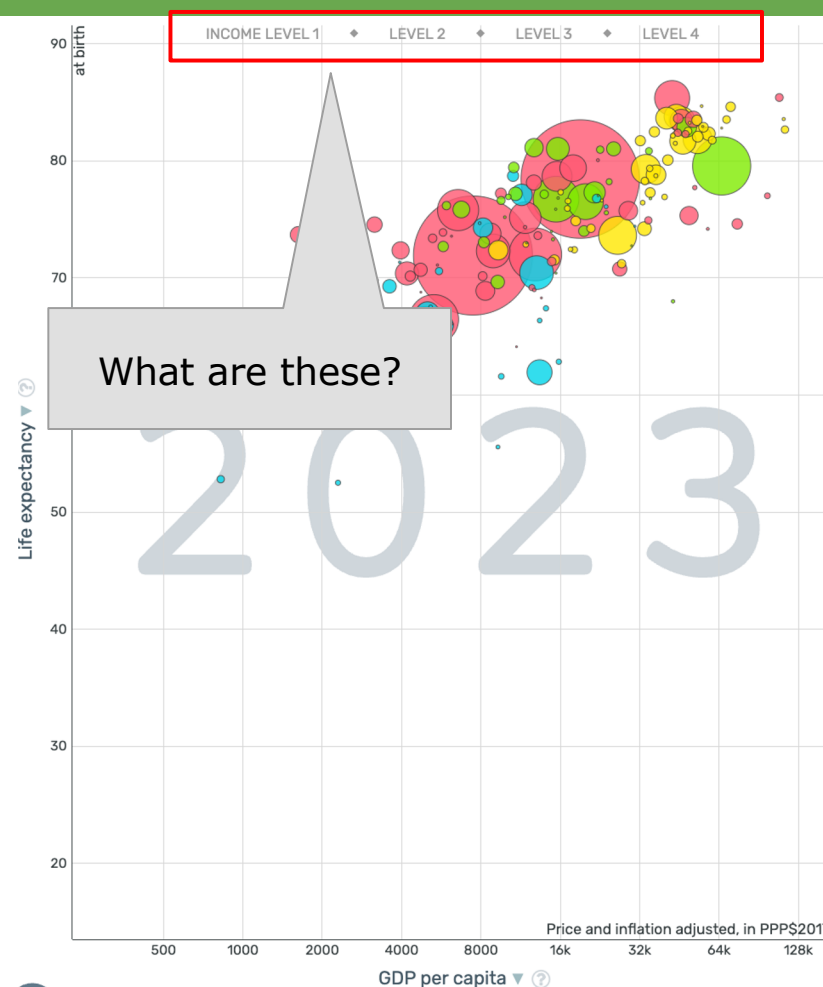
How the World Bank classify income levels

<https://blogs.worldbank.org/en/opendata/new-country-classifications-income-level-2019-2020>

Threshold	July 2019/\$ (new)	July 2018/\$ (old)
Low income	< 1,025	< 995
Lower-middle income	1,026 - 3,995	996 - 3,895
Upper-middle income	3,996 - 12,375	3,896 - 12,055
High income	> 12,375	> 12,055

Further reading:

[World Bank country classifications by income level for 2024-2025](#)



Create income-level indicator

```
df_s.loc[:, 'income_level'] = None
```

```
df_s.loc[df_s.e_gdppc < 1.025, 'income_level'] = "Low"
```

```
df_s.loc[(df_s.e_gdppc >= 1.025) & (df_s.e_gdppc < 3.995), 'income_level'] =  
"Low-middle"
```

```
df_s.loc[(df_s.e_gdppc >= 3.995) & (df_s.e_gdppc < 12.375), 'income_level'] =  
"High-middle"
```

```
df_s.loc[df_s.e_gdppc >= 12.375, 'income_level'] = "High"
```

Do you notice the
change in unit?

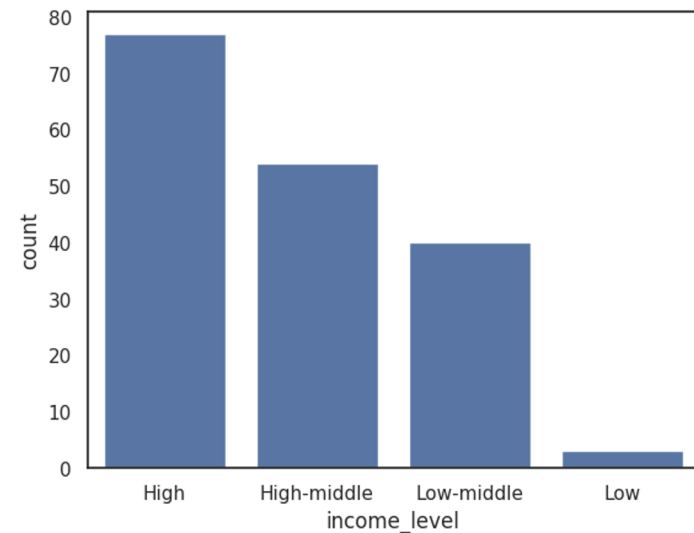
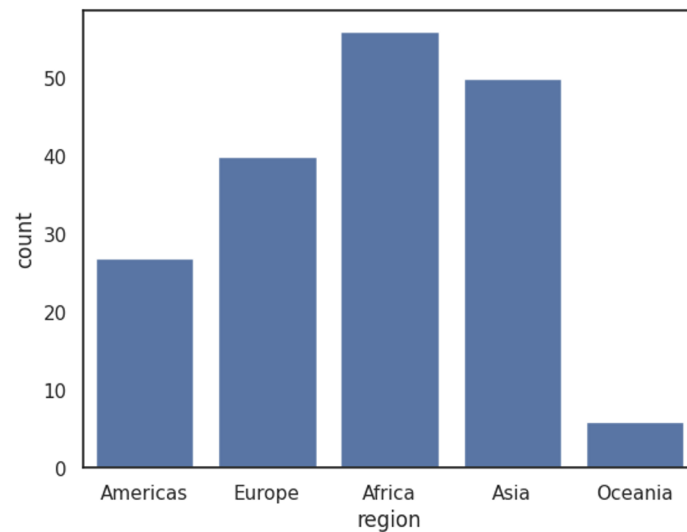
I didn't at first. Need
to learn from the
data and codebook!

1 Categorical Variable (Visualization)

Bar Chart (for count of categorical variables)

```
sns.countplot(data = df_s, x = "region")
```

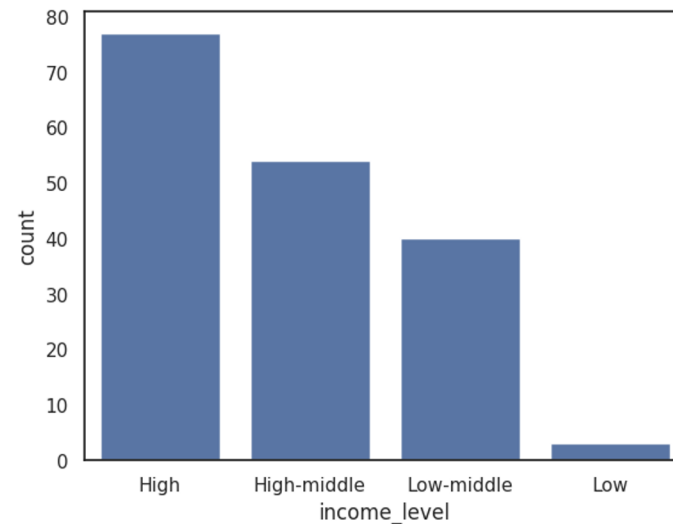
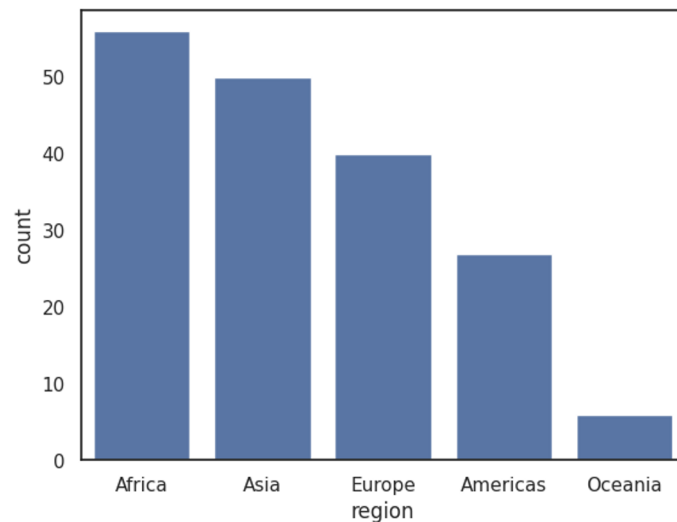
```
sns.countplot(data = df_s, x = "income_level")
```



Do it the hard way?

```
sns.barplot(df_s.region.value_counts().reset_index(), x = "region", y = "count")
```

```
sns.barplot(df_s.income_level.value_counts().reset_index(), x = "income_level", y = "count")
```



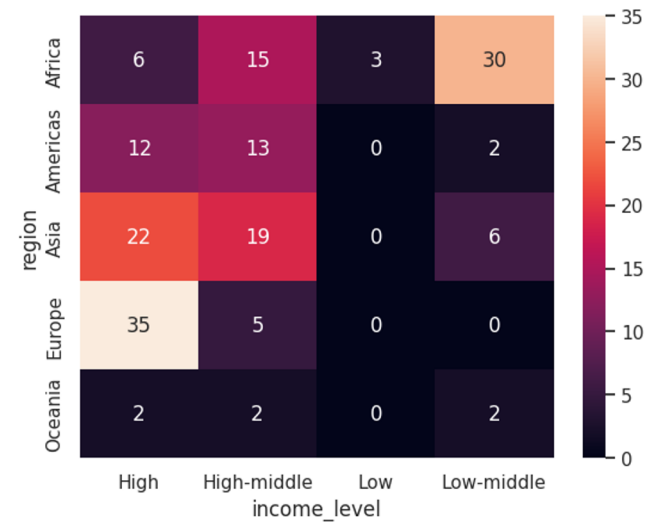
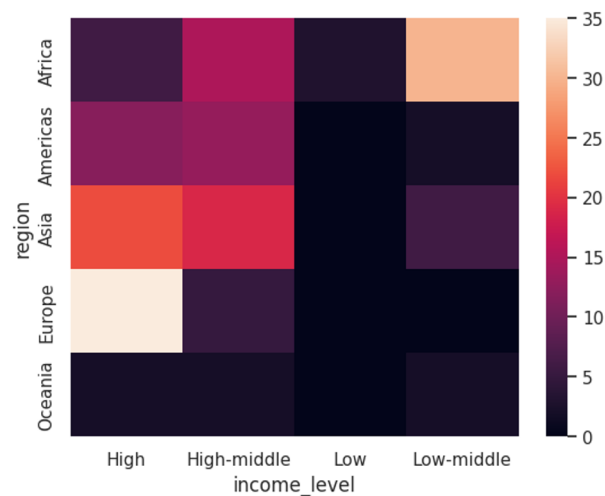
**Question for:
How to sort the levels on
the x-axis?**

2 Categorical Variables

Heatmap

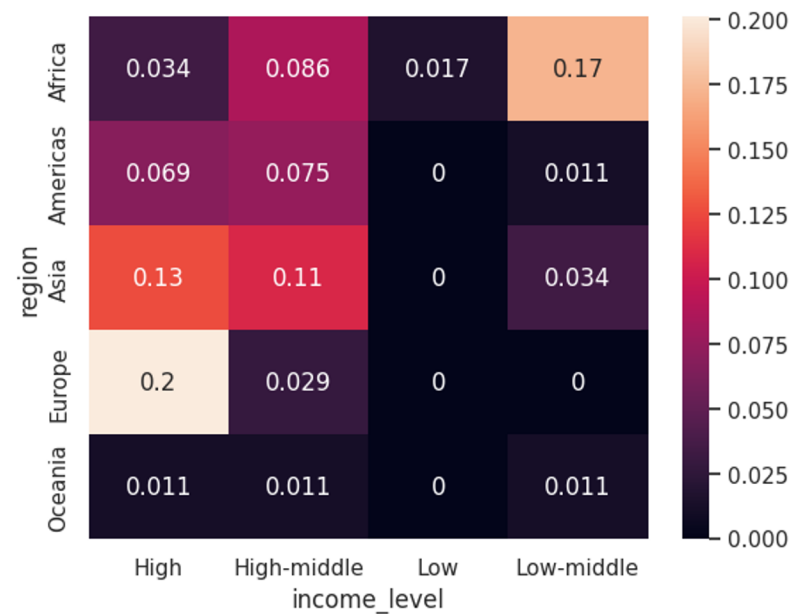
```
sns.heatmap(pd.crosstab(df_s.region, df_s.income_level))
```

```
sns.heatmap(pd.crosstab(df_s.region, df_s.income_level), annot = True)
```



Heatmap (normalized)

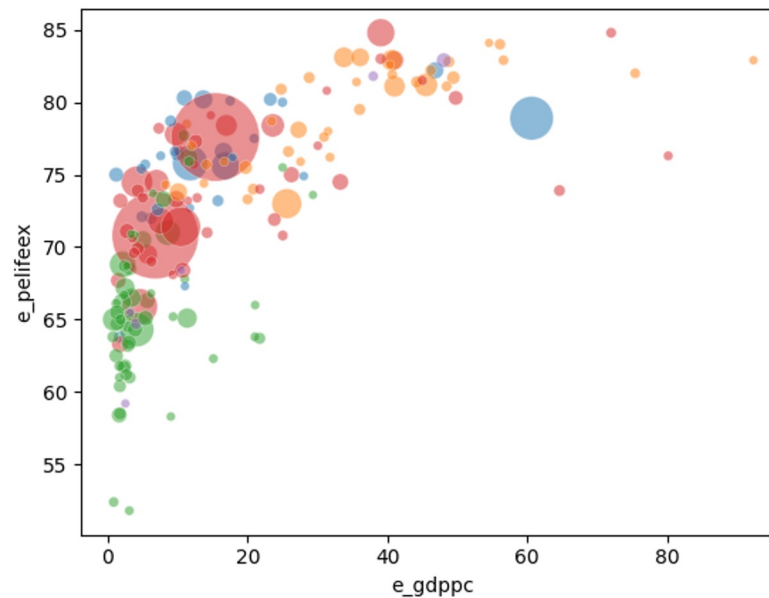
```
sns.heatmap(pd.crosstab(df_s.region, df_s.income_level, normalize = True), annot = True)
```



Quantitative Variables + 1 Categorical Variable

Use color to indicate the categorical variable (region)

```
sns.scatterplot(data = df_s, x = "e_gdppc", y = "e_pelifeex", size = 'e_pop',  
                sizes = (20, 2000), hue = 'region', legend = False, alpha = 0.5)
```



Prettify your visualization

Now that your plot gets more complicated,
It's time to talk about labeling and aesthetics...

Make *clean and informative* visualization

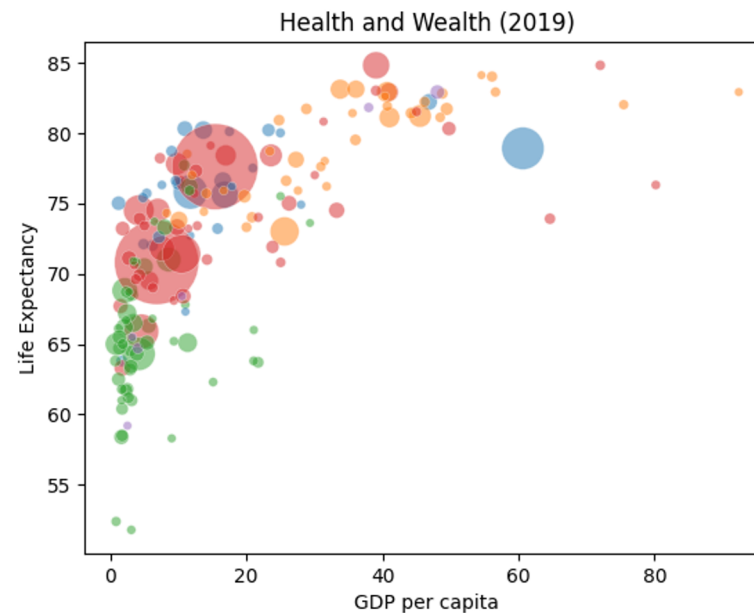
The tedious but essential: Make your figures readable

- Add the title of the figure *and* labels of x and y axes
- Adjust figure sizes
- Choose a theme you like



Title of the figure and labels of x and y axes

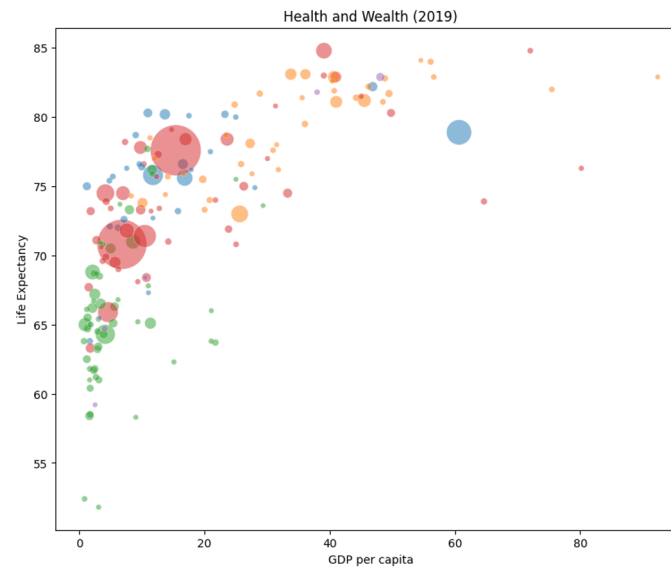
```
sns.scatterplot(data = df_s, x = "e_gdppc", y = "e_pelifeex", size = 'e_pop', sizes = (20, 2000), hue = 'region', legend = False, alpha = 0.5).set(title = "Health and Wealth (2019)", xlabel = "GDP per capita", ylabel = "Life Expectancy")
```



Adjust Size

```
plt.figure(figsize=(10, 8))
```

```
sns.scatterplot(data = df_s, x = "e_gdppc", y = "e_pelifeex", size = 'e_pop', sizes = (20, 2000), hue = 'region', legend = False, alpha = 0.5).set(title = "Health and Wealth (2019)", xlabel = "GDP per capita", ylabel = "Life Expectancy")
```



Add legends back now that we have the space?

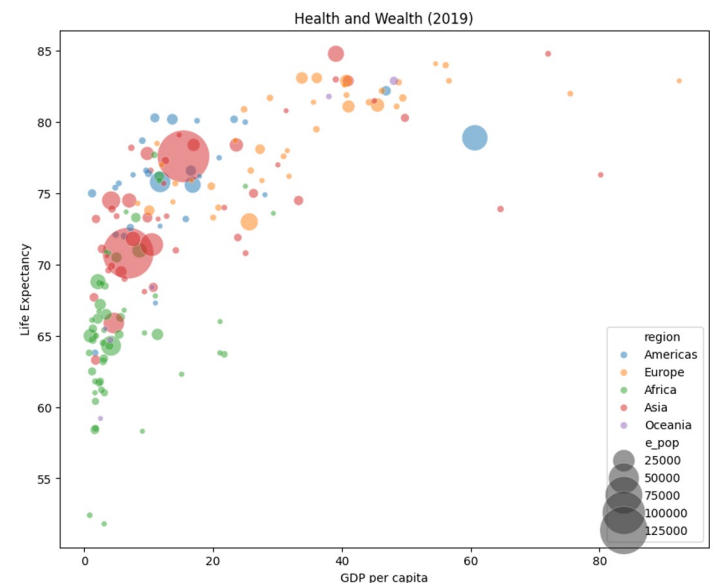
```
plt.figure(figsize=(10, 8))
```

```
sns.scatterplot(data = df_s, x = "e_gdppc", y = "e_pelifeex", size = 'e_pop', sizes = (20, 2000), hue = 'region', legend = True, alpha = 0.5).set(title = "Health and Wealth (2019)", xlabel = "GDP per capita", ylabel = "Life Expectancy")
```

Good practice: Include the legends!

Challenge: No quick and easy way to re-label the *legends*.

Suggestion: Change variable names before plot



Themes

Default is fine, but you are adventurous, try the following.

```
sns.set_style("white")
```

```
sns.set_style("whitegrid")
```

```
sns.set_style("darkgrid")
```

```
sns.set_style("dark")
```

Note: these commands have **global** impact. Once set, they impacts over every figure you plot afterwards. You typically put it at the start of a notebook.

For example: Get a darker background

```
sns.set_style("darkgrid")
```

```
plt.figure(figsize=(10, 8))
```

```
sns.scatterplot(  
    data = df_s,  
    x = "e_gdppc",  
    y = "e_pelifeeex",  
    size = 'e_pop',  
    sizes = (20, 2000),  
    hue = 'region',  
    legend = True, alpha = 0.5).  
set(  
    title = "Health and Wealth (2019)",  
    xlabel = "GDP per capita",  
    ylabel = "Life Expectancy")
```

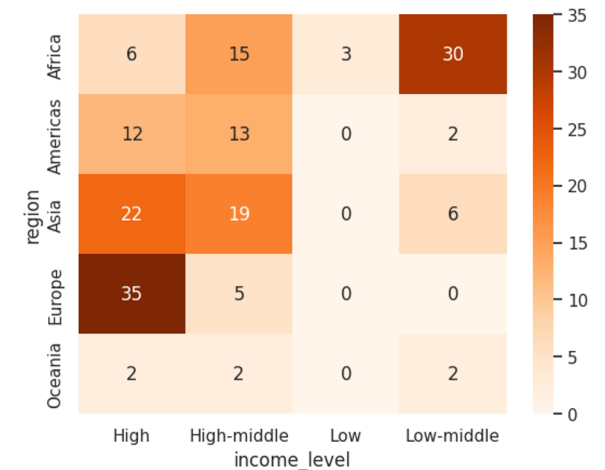
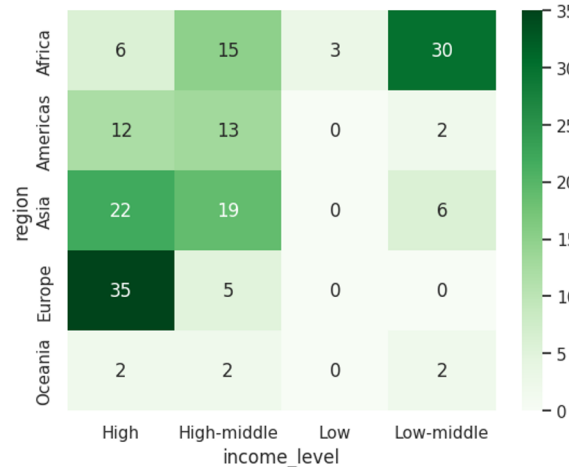
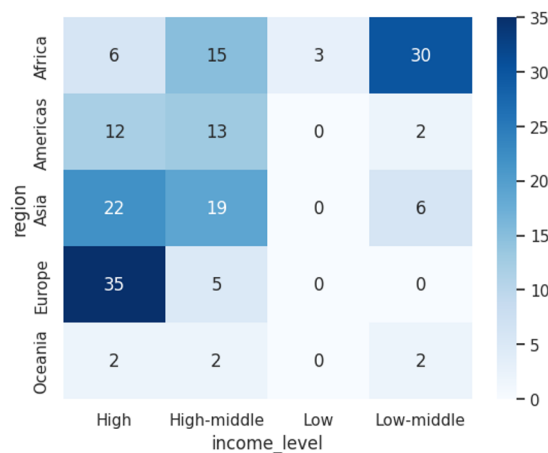


Color palette

Varies by specific plots of interest. For example:

```
sns.heatmap(pd.crosstab(df_s.region, df_s.income_level), annot = True, cmap = "Blues")
```

* try replace "Blues" with "Greens" or "Oranges" (mind the first Capital letter)



Save Figure to File

```
sns.set_style("white")  
plt.figure(figsize=(10, 8))  
sns.scatterplot(data = df_s, x = "e_gdppc", y = "e_pelifeex", size = 'e_pop',  
                sizes = (20, 2000), hue = 'region', legend = True, alpha = 0.5).  
    set(title = "Health and Wealth (2019)",  
        xlabel = "GDP per capita", ylabel = "Life Expectancy")  
plt.savefig("[YOUR PATH]/wealth_and_health_2019.png")  
plt.savefig("[YOUR PATH]/wealth_and_health_2019.pdf")
```

Further Readings

- This lecture has covered the *basics*. Make sure you understand all.
- Other visualization are mostly variants of of these basic types
- Read the full official Seaborn tutorial:
<https://seaborn.pydata.org/tutorial/introduction.html>
- Learn what visualization are available:
<https://python-graph-gallery.com/>
 - Specifically about Seaborn: <https://python-graph-gallery.com/seaborn/>
- Use the search engine
- *Reminder:*
 - Getting the concepts and designs right is more important than specific techniques/ tools
 - There are multiple ways to make the same visualization. Choose wisely
 - Search engines and AI may give you suboptimal suggestions!